

Explain Happy Strings

1. Big Picture

This lesson is about **LeetCode 1415 – The k-th Lexicographical String of All Happy Strings of Length n**.

The main idea is a classic **backtracking** pattern:

```
Choose → Explore → Undo
```

The problem asks us to generate strings of length `n` using only:

```
'a', 'b', 'c'
```

with one restriction:

```
No two adjacent characters can be equal.
```

For example:

```
aba → happy
abc → happy
aca → happy
```

But:

```
aab → not happy
abb → not happy
ccc → not happy
```

The important observation from the video is that if we always try characters in this order:

```
a → b → c
```

then our backtracking automatically generates the valid strings in **lexicographical order**. Therefore, we don't need to generate everything and sort it afterward.

The video ultimately teaches **two approaches**:

- 1. Generate all happy strings → store them → return the $(k-1)$ th one.
- 2. Generate them in lexicographical order → count them → stop immediately when we reach the k th string.

The second approach is the important optimization.

2. Prerequisites

You should understand these concepts before this problem:

2.1 Recursion

A function calls itself to solve a smaller version of the same problem.

For example:

```
solve(...);
```

inside `solve()`.

2.2 Backtracking

Backtracking means:

```
DO
↓
EXPLORE
↓
UNDO
```

For a string:

```
current = ""
```

we may:

```
current = "a"
```

explore everything starting with "a" ,

then undo:

```
current = ""
```

and try:

```
current = "b"
```

This is the central pattern of the problem.

2.3 Lexicographical Order

For strings, lexicographical order is essentially dictionary order.

For example:

```
aba
abc
aca
acb
bab
bac
...
```

Because:

```
a < b < c
```

So whenever we have a choice, trying:

```
a → b → c
```

causes smaller strings to be generated before larger strings.

2.4 Zero-Based Indexing

If the user asks for the:

```
k-th
```

string, its vector index is:

```
k - 1
```

For example:

```
k = 1 → index 0
k = 2 → index 1
k = 3 → index 2
```

3. Detailed Explanation

3.1 What Exactly Is a Happy String?

A happy string has two conditions.

Condition 1 — Only a, b, c

For example:

```
abc
aba
cac
```

are allowed.

Something like:

```
abd
```

is invalid because d is not allowed.

Condition 2 — Adjacent characters cannot be equal

For every adjacent pair:

```
s[i] != s[i + 1]
```

For example:

```
aba
```

Check:

```
a != b  
b != a
```

So it is happy.

But:

```
abb
```

has:

```
b == b
```

so it is not happy.

3.2 What Is the Problem Asking?

Given:

```
n  
k
```

we need to:

1. Generate every happy string of length `n`.
2. Arrange them in lexicographical order.
3. Return the `k`th string.
4. If fewer than `k` happy strings exist, return:

```
""
```

3.3 Simple Example

Suppose:

```
n = 1
```

Possible strings:

```
a  
b  
c
```

They are already lexicographically sorted.

So:

```
k = 1 → "a"  
k = 2 → "b"  
k = 3 → "c"  
k = 4 → ""
```

Why "" for `k = 4`?

Because only three happy strings exist.

3.4 Example: `n = 2`

Let's generate all possibilities.

At the first position:

```
a  
b  
c
```

Suppose we choose `a`.

The next character cannot be a .

Therefore:

```
ab
ac
```

Similarly, after b :

```
ba
bc
```

After c :

```
ca
cb
```

Therefore the complete list is:

```
ab
ac
ba
bc
ca
cb
```

Notice something important:

There are:

```
3 x 2 = 6
```

happy strings.

3.5 How Many Happy Strings Exist?

This is an important observation.

For the first position:

```
a
b
c
```

There are:

```
3 choices
```

For every position after that, we cannot use the previous character.

So there are only:

```
2 choices
```

For example:

```
a → b/c
b → a/c
c → a/b
```

Therefore:

```
Number of happy strings
= 3 x 2 x 2 x ... x 2
```

There are $n - 1$ occurrences of 2 .

Therefore:

$3 \times 2^{n-1}$

happy strings exist.

For:

```
n = 3
```

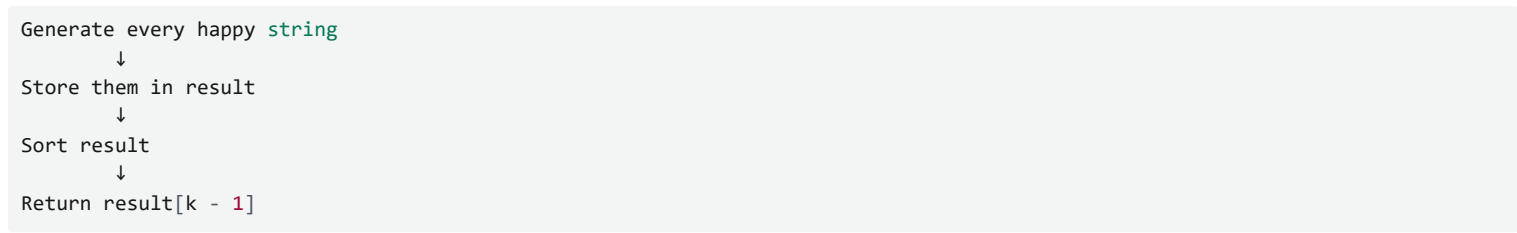
we get:

$$3 \times 2^2 = 12$$

So there are exactly 12 happy strings.

3.6 First Approach — Generate Everything and Sort

The most straightforward idea is:



For example:

```
n = 3
```

We might generate:

```
aba
abc
aca
acb
...
```

Even if the generation order were random, we could do:

```
sort(result.begin(), result.end());
```

and then:

```
return result[k - 1];
```

This works.

But there is unnecessary work.

3.7 Why Sorting Is Unnecessary

Backtracking gives us control over the order in which we explore possibilities.

Suppose:

```
current = "a"
```

For the next character we try:

```
a
b
c
```

But `a` is invalid because:

```
current.back() == 'a'
```

So we try:

```
b
```

before:

```
c
```

Thus we generate:

```
ab...
```

before:

```
ac...
```

Likewise, all strings beginning with `a` are generated before strings beginning with `b`, and all strings beginning with `b` before strings beginning with `c`.

Therefore:

If we always explore `a`, then `b`, then `c`, the DFS traversal itself produces the strings in lexicographical order.

This is the key insight.

3.8 The Backtracking Tree

Let's use:

`n = 3`

At the beginning:

`""`

We have three choices:



Let's explore the `a` branch first.

After choosing `a`, we cannot choose `a` again:



From `ab`, we cannot choose `b`:



So the beginning of our traversal is:

`aba`
`abc`
`aca`
`acb`
`...`

This is already lexicographical order.

3.9 Why DFS Gives Lexicographical Order

This is worth understanding deeply.

Imagine a dictionary.

All words beginning with:

`a`

come before words beginning with:

`b`

and all words beginning with:

`b`

come before words beginning with:

`c`

Our recursion behaves exactly like that.

We completely explore:

`a...`

before moving to:

`b...`

and completely explore:

```
b...
```

before moving to:

```
c...
```

Inside the `a` branch, we again try:

```
b
```

before:

```
c
```

So the ordering happens naturally.

3.10 The Backtracking Template

The instructor repeatedly emphasizes the standard backtracking family template:

```
DO
↓
EXPLORE
↓
UNDO
```

For this problem:

DO

Add a character:

```
current.push_back(ch);
```

EXPLORE

Recursively continue:

```
solve(...);
```

UNDO

Remove the character:

```
current.pop_back();
```

So the core pattern becomes:

```
current.push_back(ch);

solve(...);

current.pop_back();
```

This pattern appears in many backtracking problems.

3.11 Base Case

We need to know when we have constructed a complete happy string.

If:

```
current.length() == n
```

then we have finished.

For example:

```
n = 3

current = "aba"
```

Its length is 3.

Therefore:

```
current.length() == n
```

is true.

At this point, "aba" is a complete answer.

3.12 First Implementation

A clean version of the first approach is:

```
class Solution {
public:

    void solve(int n, string &current, vector<string> &result) {

        // Base case
        if (current.length() == n) {
            result.push_back(current);
            return;
        }

        // Try a, b, c
        for (char ch = 'a'; ch <= 'c'; ch++) {

            // Adjacent characters cannot be equal
            if (!current.empty() && current.back() == ch) {
                continue;
            }

            // DO
            current.push_back(ch);

            // EXPLORE
            solve(n, current, result);

            // UNDO
            current.pop_back();
        }
    }

    string getHappyString(int n, int k) {

        string current;
        vector<string> result;

        solve(n, current, result);

        if (result.size() < k) {
            return "";
        }

        return result[k - 1];
    }
};
```

3.13 Code Explanation

```
string current
```

```
string current;
```

This stores the string we are currently constructing.

Initially:

```
current = ""
```

Then:

```
"a"
"ab"
"aba"
```

and so on.

```
vector<string> result
```

```
vector<string> result;
```

This stores every completed happy string.

For example:

```
result = {  
    "aba",  
    "abc",  
    "aca",  
    "acb",  
    ...  
}
```

Base Case

```
if (current.length() == n) {  
    result.push_back(current);  
    return;  
}
```

Meaning:

```
"I have constructed a complete string of the required length. Store it and stop exploring this branch."
```

The Loop

```
for (char ch = 'a'; ch <= 'c'; ch++)
```

This produces:

```
'a'  
'b'  
'c'
```

in exactly that order.

That's what gives us lexicographical ordering.

Checking the Previous Character

```
if (!current.empty() && current.back() == ch) {  
    continue;  
}
```

There are two things happening here.

```
!current.empty()
```

We must make sure the string isn't empty.

Why?

Because:

```
current.back()
```

on an empty string is invalid.

```
current.back() == ch
```

This checks whether the new character is equal to the previous character.

Suppose:

```
current = "ab"
```

and:

```
ch = 'b'
```

Then:

```
current.back() = 'b'
```

So:

```
'b' == 'b'
```

is true.

Therefore we:

```
continue;
```

and skip this choice.

3.14 Why `continue`?

Suppose:

```
current = "ab"
```

We're trying:

```
a  
b  
c
```

For a:

```
b != a
```

Allowed.

For b:

```
b == b
```

Not allowed.

So:

```
continue;
```

means:

```
Don't use this character. Move to the next loop iteration.
```

Then we try:

```
c
```

which is valid.

3.15 DO → EXPLORE → UNDO

Suppose:

```
current = "ab"
```

and we choose:

```
c
```

DO

```
current.push_back('c');
```

Now:

```
current = "abc"
```

EXPLORE

```
solve(...);
```

Since the length is now 3, we reach the base case.

UNDO

After returning:

```
current.pop_back();
```

So:

```
"abc"
```

becomes:

```
"ab"
```

This is extremely important.

We remove `c` because we want to try another possibility from `"ab"`.

3.16 Full Dry Run

Let's use:

```
n = 3
```

Initially:

```
current = ""
```

Try:

```
a
```

Now:

```
current = "a"
```

Try `a`.

Invalid because:

```
a == a
```

Try `b`.

Valid:

```
current = "ab"
```

Try `a`.

Valid:

```
current = "aba"
```

Length is 3.

Store:

```
result = ["aba"]
```

Return.

Undo:

```
current = "ab"
```

Now try `b`.

Invalid:

```
b == b
```

Try `c`.

Valid:

```
current = "abc"
```

Store:

```
result = [
  "aba",
  "abc"
]
```

Return.

Undo:

```
current = "ab"
```

No more choices.

Return again.

Undo b:

```
current = "a"
```

Now try c.

```
current = "ac"
```

From "ac":

Try a:

```
aca
```

Store.

Then:

```
acb
```

Store.

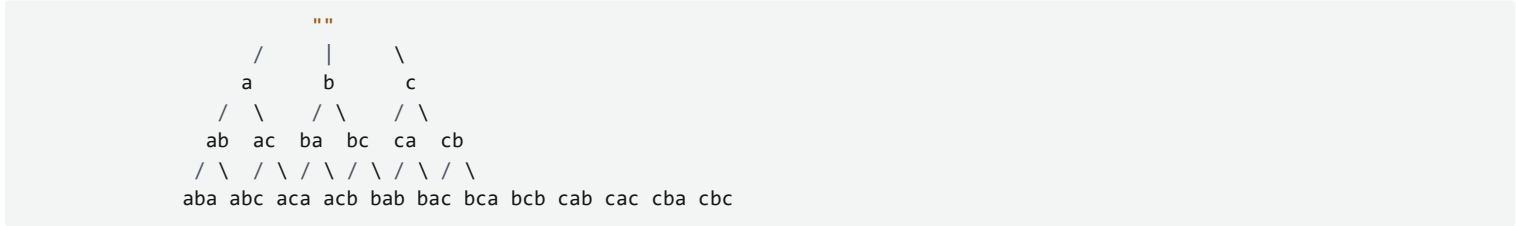
So our result starts:

```
aba
abc
aca
acb
...
```

Notice that it is already sorted.

3.17 Complete Tree for n = 3

The important part looks like:



Therefore:

```
1  → aba
2  → abc
3  → aca
4  → acb
5  → bab
6  → bac
7  → bca
8  → bcb
9  → cab
10 → cac
11 → cba
12 → cbc
```

So:

```
n = 3
k = 9
```

gives:

```
"cab"
```

3.18 Why the Second Approach Is Better

The first approach stores:

```
all happy strings
```

But the question only asks for:

```
one string → the kth string
```

Suppose:

```
n = 10
```

There can be:

$$3 \times 2^9 = 1536$$

happy strings.

If we only need the 5th one, storing all 1536 strings is unnecessary.

We can instead count:

```
1
2
3
4
5
```

and stop immediately.

This is the optimization introduced in the video.

3.19 Optimized Approach

Instead of:

```
vector<string> result;
```

we maintain:

```
int count = 0;
```

Whenever we construct a complete happy string:

```
count++;
```

Then:

```
if (count == k)
```

we have found the answer.

Store only that string.

3.20 Why Does This Work?

Because our DFS already generates strings in lexicographical order.

Suppose the generated order is:

```
1. aba
2. abc
3. aca
4. acb
5. bab
...
```

If:

```
k = 4
```

we don't need:

```
aba  
abc  
aca
```

after counting them.

We only need:

```
acb
```

So:

```
count++;
```

and when:

```
count == k
```

we save:

```
result = current;
```

and stop.

3.21 Optimized Code

```
class Solution {  
public:  
  
    void solve(  
        int n,  
        string &current,  
        int &count,  
        int k,  
        string &result  
    ) {  
  
        // Base case  
        if (current.length() == n) {  
  
            count++;  
  
            // We found the kth happy string  
            if (count == k) {  
                result = current;  
            }  
  
            return;  
        }  
  
        // Try a, b, c  
        for (char ch = 'a'; ch <= 'c'; ch++) {  
  
            // Adjacent characters cannot be equal  
            if (!current.empty() && current.back() == ch) {  
                continue;  
            }  
  
            // DO  
            current.push_back(ch);  
  
            // EXPLORE  
            solve(n, current, count, k, result);  
  
            // If answer is already found, stop exploring  
            if (!result.empty()) {  
                return;  
            }  
        }  
    }  
};
```

```

        // UNDO
        current.pop_back();
    }
}

string getHappyString(int n, int k) {

    string current;
    string result;

    int count = 0;

    solve(n, current, count, k, result);

    return result;
}
};

```

There is one subtle implementation detail here: if we return from the recursive function before `pop_back()`, the current string in the caller remains modified. That's harmless here because we're immediately terminating the entire search path, but a cleaner version can preserve the undo invariant by checking the result after recursion and popping before returning.

I recommend writing it like this:

```

class Solution {
public:

    void solve(
        int n,
        string &current,
        int &count,
        int k,
        string &result
    ) {

        if (current.length() == n) {

            count++;

            if (count == k) {
                result = current;
            }

            return;
        }

        for (char ch = 'a'; ch <= 'c'; ch++) {

            if (!current.empty() && current.back() == ch) {
                continue;
            }

            // DO
            current.push_back(ch);

            // EXPLORE
            solve(n, current, count, k, result);

            // UNDO
            current.pop_back();

            // Stop once kth string is found
            if (!result.empty()) {
                return;
            }
        }
    }

    string getHappyString(int n, int k) {

        string current;
        string result;

        int count = 0;

```

```
        solve(n, current, count, k, result);

    return result;
}
};
```

This version maintains the clean:

```
DO
EXPLORE
UNDO
```

structure.

3.22 Execution of the Optimized Version

Consider:

```
n = 3
k = 4
```

The DFS produces:

```
aba
abc
aca
acb
```

At:

```
aba
```

we do:

```
count = 1
```

At:

```
abc
```

we do:

```
count = 2
```

At:

```
aca
```

we do:

```
count = 3
```

At:

```
acb
```

we do:

```
count = 4
```

Now:

```
count == k
```

so:

```
result = "acb";
```

We stop searching.

We don't generate:

```
bab
bac
bca
bcb
cab
...
```

because they are irrelevant.

3.23 What If `k` Is Too Large?

Suppose:

```
n = 1
k = 4
```

There are only:

```
a
b
c
```

So:

```
count
```

will reach only:

```
3
```

It will never become:

```
4
```

Therefore:

```
result = ""
```

and we return:

```
""
```

This automatically handles the invalid `k` case.

3.24 Complexity

There are:

$3 \times 2^{n-1}$

possible happy strings.

Each string has length:

n

So in the worst case, if we have to generate almost all strings:

$O(n \times 3 \times 2^{n-1})$

Ignoring constants:

$O(n2^n)$

Why the extra `n`?

Because when we construct/copy a complete string of length `n`, that operation takes up to:

```
O(n)
```

Space Complexity

The recursion depth is at most:

```
n
```

and `current` contains at most:

n

characters.

Therefore auxiliary space is:

$O(n)$

This is the major improvement over storing every generated string.

3.25 First Approach vs Optimized Approach

Feature	Approach 1	Approach 2
Generate strings	Yes	Yes
Lexicographical generation	Yes	Yes
Store all strings	Yes	No
Sort required	Not if generated lexicographically	No
Counter	No	Yes
Stops at kth string	No	Yes
Extra result storage	$O(n2^n)$	$O(n)$
Worst-case time	$O(n2^n)$	$O(n2^n)$
Worst-case recursion space	$O(n)$	$O(n)$

The key point:

The optimization mainly reduces space, and can also reduce actual runtime when k is small because we stop early.

The worst-case time complexity remains exponential.

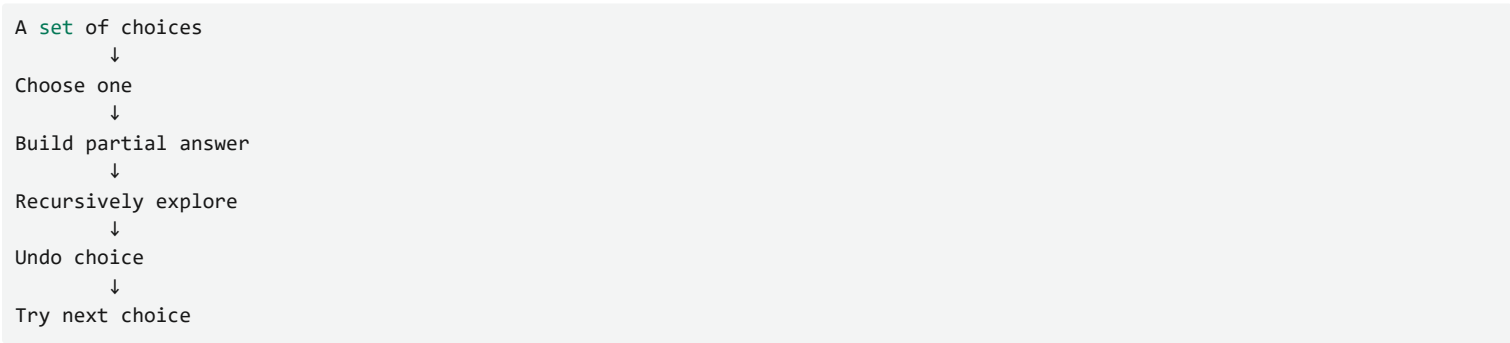
3.26 The Most Important Backtracking Insight

Don't memorize this problem as:

"LeetCode 1415 requires this particular code."

Instead, recognize the pattern.

We have:



Here the choices are:

- a

b

c

with the constraint:

new character != previous character

So:

```
for (char ch = 'a'; ch <= 'c'; ch++) {

    if (!current.empty() && current.back() == ch)
        continue;

    current.push_back(ch);

    solve(...);

    current.pop_back();
}
```

That is the real reusable template.

3.27 Why We Don't Need `used[]`

This is another important point.

In permutation problems, you often see:

```
vector<bool> used(n);
```

because you need to remember which elements have already been used.

We don't need that here.

Why?

Because characters can be reused.

For example:

```
aba
```

contains a twice.

The only restriction is:

```
current character != immediately previous character
```

Therefore we only need:

```
current.back()
```

rather than a complete `used[]` array.

3.28 Why Can We Reuse a Character?

Suppose:

```
current = "ab"
```

Can we choose a?

Yes.

We get:

```
aba
```

There is no rule saying:

```
"You cannot use the same character again."
```

The rule only says:

```
"You cannot use the same character immediately next to itself."
```

So:

```
aba
```

is valid.

But:

```
abb
```

is invalid.

This distinction is crucial.

3.29 Connection to Other Backtracking Problems

This problem belongs to the same general family as:

Subsets

At every position:

```
take / don't take
```

Permutations

At every position:

```
choose one unused element
```

Combination Sum

At every step:

```
choose a candidate
```

while satisfying a target constraint.

N-Queens

At every row:

```
choose a valid column
```

while satisfying constraints.

This problem

At every position:

```
choose a / b / c
```

while satisfying:

```
new character != previous character
```

The underlying skeleton is the same.

3.30 A Better Mental Model

Think of the recursion as **building a path through a decision tree**.

At every node:

```
What choices do I have?
```

Here:

```
a
b
c
```

Then:

```
Which choices are invalid?
```

Only the character equal to the previous one.

Then:

```
Choose one valid option.
```

Then:

```
Go deeper.
```

Then:

```
Come back and try another option.
```

That's backtracking.

4. Key Takeaways

1. A happy string

Uses only:

a, b, c

and satisfies:

`s[i] != s[i+1]`

2. Number of happy strings

For length `n`:

$$3 \times 2^{n-1}$$

because:

first position → 3 choices
every next position → 2 choices

3. Backtracking template

Remember:

DO
↓
EXPLORE
↓
UNDO

In code:

```
current.push_back(ch);  
  
solve(...);  
  
current.pop_back();
```

4. Lexicographical order

Try:

a → b → c

This causes DFS to generate the answers in lexicographical order.

Therefore, **sorting is unnecessary**.

5. Constraint checking

We only need to inspect the previous character:

```
if (!current.empty() && current.back() == ch)  
    continue;
```

No `used[]` array is required.

6. First solution

Generate and store all strings:

generate → store → return `result[k-1]`

7. Better solution

Generate in lexicographical order and maintain:

count

When:

```
count == k
```

save the current string and stop.

8. Complexity

Worst case:

 $O(n2^n)$

Space:

 $O(n)$

for the optimized approach, excluding the returned string itself.

5. Common Mistakes / Misunderstandings

Mistake 1: Thinking repeated characters are completely forbidden

Wrong:

```
aba → invalid
```

Correct:

```
aba → valid
```

Only **adjacent** equal characters are forbidden.

Mistake 2: Using `used[]`

You don't need it.

Characters can be reused.

You only need to know:

```
current.back()
```

Mistake 3: Sorting the result unnecessarily

If your DFS does:

```
a → b → c
```

then it already generates strings lexicographically.

Sorting afterward is unnecessary.

Mistake 4: Forgetting `pop_back()`

This is a classic backtracking bug.

Without:

```
current.pop_back();
```

the next branch will inherit the previous choice.

For example, instead of:

```
ab
ac
```

you could accidentally build something like:

```
abc...
```

with the old state still present.

Mistake 5: Calling `current.back()` on an empty string

This is unsafe:

```
if (current.back() == ch)
```

when:

```
current = ""
```

Use:

```
if (!current.empty() && current.back() == ch)
```

Mistake 6: Returning `result[k]`

Vectors use zero-based indexing.

If:

```
k = 3
```

the third element is:

```
result[2]
```

So:

```
result[k - 1]
```

Mistake 7: Thinking the optimized solution becomes polynomial

It doesn't.

The number of possible happy strings is still exponential:

$$3 \times 2^{n-1}$$

The optimization mainly avoids **storing all answers** and can terminate early.

6. Quick Revision

If you need to remember this problem in an interview, remember this:

Problem:

Find kth lexicographical happy `string` of length `n`.

Characters:

a, b, c

Constraint:

Adjacent characters cannot be equal.

Backtracking:

```
for ch = 'a' to 'c'
```

```
    if ch == previous:
        skip
```

```
    push ch
```

```
    recurse
```

```
    pop ch
```

Why lexicographical?

Because we always try:

a → b → c

Base case:

```
current.length() == n
```

Optimization:

Don't store every string.

Maintain count.

When:

```
count == k
```

```
answer = current
```

```
stop.
```

Number of possibilities:

$$3 \times 2^{(n-1)}$$

Worst-case time:

$O(n \times 2^n)$

Space:

$O(n)$

7. Questions You Should Be Able to Answer

Before considering this problem mastered, you should be able to answer these without looking at the code:

1. What makes a string a happy string?
2. Why are there $3 \times 2^{(n-1)}$ possible happy strings?
3. Why do we only need to check the previous character?
4. Why don't we need a `used[]` array?
5. Why does trying `a → b → c` automatically produce lexicographical order?
6. What exactly is the base case?
7. Why do we call `pop_back()` after recursion?
8. What does `current` represent at any moment in the recursion?
9. What is the difference between the first and optimized approaches?
10. Why does the optimized approach use `count` instead of `vector<string> result`?
11. Why can the optimized approach stop as soon as `count == k`?
12. Why is the worst-case time still $O(n2^n)$?
13. Why is the auxiliary space $O(n)$?
14. What would happen if we removed `pop_back()`?
15. What would happen if we tried `b` before `a`?
16. For `n = 2`, can you manually generate all happy strings in lexicographical order?
17. For `n = 3`, `k = 9`, can you explain why the answer is `cab`?
18. Can you write the backtracking template without looking at the solution?

The most important thing to take away is not LeetCode 1415 itself. It is recognizing this pattern:

```
CHOICES
↓
Check constraint
↓
DO / PUSH
↓
DFS
↓
UNDO / POP
↓
Try next choice
```

Once you see a problem that says "**generate all possible valid sequences**", especially when there are constraints on each choice, **backtracking should immediately come to mind**. The transcript itself frames this problem as another member of the same backtracking family and builds the solution around that template.